

Neural Introspection Gating for Adaptive KV-Cache Reuse in Vision-Language-Action Models

Zhijie Wu¹, Kento Kawaharazuka¹, Kei Okada¹

Abstract—Vision-Language-Action (VLA) models map camera images and language instructions directly to motor commands through a single autoregressive transformer. In real-time control, they still spend substantial compute recomputing key-value (KV) representations for visual tokens that barely change across neighboring frames. Recent work such as VLA-Cache reduces that cost by reusing KV states for visually static patches, but its policy relies only on observation-space heuristics and does not account for the model’s own uncertainty. We propose Gated VLA-Cache, a lightweight, training-free extension that augments visual-similarity caching with *neural introspection*. The method monitors the logit margin between the top-two predicted action tokens, a zero-cost confidence signal available during decoding. When the margin drops below a threshold, the cache is invalidated and a full recompute is triggered. Evaluated on four LIBERO benchmark suites with both OpenVLA and OpenVLA-OFT, Gated VLA-Cache improves reliability when blind caching hurts. On LIBERO-Goal and LIBERO-Long, it recovers over 100% of the lost accuracy while retaining 80% of the compute savings.

I. INTRODUCTION

Vision-Language-Action (VLA) models [1]–[3] use a single autoregressive transformer that ingests a camera frame and a language instruction, then directly outputs discretized motor commands. Because they build on large vision-language models (VLMs), VLAs generalize across manipulation tasks without much task-specific engineering. However, running a 7-billion-parameter model at the 10–20 Hz rates needed for manipulation is computationally expensive.

Consecutive camera frames in a manipulation episode are largely redundant. VLA-Cache [4] exploits this by identifying visually static image patches via pixel-level cosine similarity, reusing their cached KV representations from the previous forward pass, and modulating per-layer reuse proportions through an entropy-adaptive schedule.

The problem is that VLA-Cache’s reuse decision is based entirely on observation-space heuristics (patch similarity and attention-weighted task relevance). It cannot tell when the model itself is uncertain, e.g., during grasp alignment or when a visually small change turns out to be behaviorally important. When that happens, stale cached representations inject errors into the autoregressive decoding of the action vector, and those errors accumulate over long horizons.

We address this gap with **Gated VLA-Cache**, a training-free extension that adds *neural introspection gating*, using the model’s own action-prediction confidence to govern cache validity (Fig. 1).

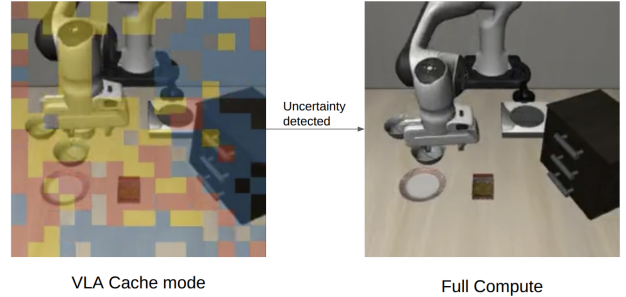


Fig. 1: **Overview.** VLA-Cache decides cache reuse based on observation-space patch similarity alone, risking stale representations at critical moments. **Blue:** static tokens, **Yellow:** task-relevant, **Red:** overlapping. Gated VLA-Cache additionally inspects the model’s own logit margin to adaptively invalidate the cache when the model’s confidence drops.

logit margin predicts action reliability. During autoregressive action decoding, the model produces a softmax distribution over action tokens at each step. A large gap between the top-1 and top-2 token probabilities means the model is confident and cache reuse is safe. A narrow margin means the model is hesitant between two actions, and a full recompute is more likely to correct the trajectory.

On the challenging LIBERO-Long suite with OpenVLA, Gated VLA-Cache fully recovers the accuracy lost by blind caching (54.8% vs. 50.2% for VLA-Cache, 54.0% for full inference) at a modest compute increase (1.54 vs. 1.43 TFLOPs), remaining 18% below full inference cost (1.88 TFLOPs).

II. RELATED WORK

Vision-Language-Action Models. VLA [1]–[3], [5] models unify perception, language understanding, and action prediction within a single transformer. RT-2 [2] pioneered the approach by fine-tuning a VLM to output actions as text tokens. OpenVLA [1] is a 7B-parameter model built on Prismatic VLMs [6] that performs well on diverse manipulation benchmarks. OpenVLA-OFT [5] further improves fine-tuning efficiency. π_0 [3] achieves strong results with a flow matching [7] head. These models typically adopt large LLM backbones [8] and are fine-tuned on large robot datasets [9], but remain too expensive to run in real time on most robotic hardware.

Efficient VLA Inference. Several approaches reduce the computational cost of VLA inference. TinyVLA [10] uses

¹ The authors are with Graduate School of Information Science and Technology, The University of Tokyo, 7-3-1 Hongo, Bunkyo-ku, Tokyo, 113-8656, Japan. [z-wu, kawaharazuka, k-okada]@jsk.imi.i.u-tokyo.ac.jp

a 422M-parameter model with a diffusion policy [11] head for lower latency. SmolVLA [12] is a 450M-parameter VLA with a flow matching [7] head. RoboMamba [13] replaces the Transformer [14] backbone with Mamba [15] for faster inference. However, these methods require retraining, and their smaller model size limits generalization to new environments.

KV-Cache Optimization for Transformers. Key-value caching is standard in autoregressive generation to avoid redundant computation. For VLMs and VLAs, several works explore efficient vision-token handling: FastV [16] prunes less important vision tokens after a designated layer; PyramidDrop [17] progressively drops tokens across layers. VLA-Cache [4] introduces a training-free caching mechanism that detects unchanged visual patches between frames via cosine similarity, selectively reuses their KV representations, and employs an entropy-adaptive layer-wise reuse schedule. Our work builds directly on VLA-Cache and extends it with a model-intrinsic gating signal that detects when cached representations are no longer safe to reuse.

Uncertainty Estimation in Neural Networks. Using model uncertainty to inform downstream decisions has a long history [18]–[20]. Margin-based confidence measures [21] are widely used in active learning and selective prediction. We adapt this idea to VLA inference, using the logit margin, available as a zero-cost byproduct of the forward pass, to gate temporal cache reuse.

III. BACKGROUND: VLA-CACHE

We briefly review the VLA-Cache [4] pipeline upon which our method builds.

A. VLA Inference

A VLA model takes as input a camera image $I_t \in \mathbb{R}^{H \times W \times 3}$ and a language instruction ℓ , and autoregressively generates a sequence of D action tokens $a_t = (a_t^1, \dots, a_t^D)$. The image is divided into $N = (H/p)(W/p)$ non-overlapping patches of size $p \times p$ (typically $p = 14$, $N = 256$ for 224×224 images), projected into token embeddings, and concatenated with language tokens before being fed into the transformer backbone. The prompt-encoding forward pass through all L layers for all $N + |\ell|$ input tokens dominates inference time.

B. VLA-Cache Pipeline

VLA-Cache reduces this cost through three stages:

Stage 1: Static Patch Detection. Given I_t and I_{t-1} , cosine similarity is computed between corresponding image patches. Patches with similarity above a threshold τ (default 0.996) are deemed static and marked as candidates for KV reuse.

Stage 2: Task-Relevant Filtering. Static patches that receive high text-to-vision attention are removed from the candidate set, since they are semantically important and should be freshly computed. The remaining patches form the reusable set.

Stage 3: Entropy-Adaptive Layer-Wise Reuse. Different transformer layers exhibit varying attention entropy, which

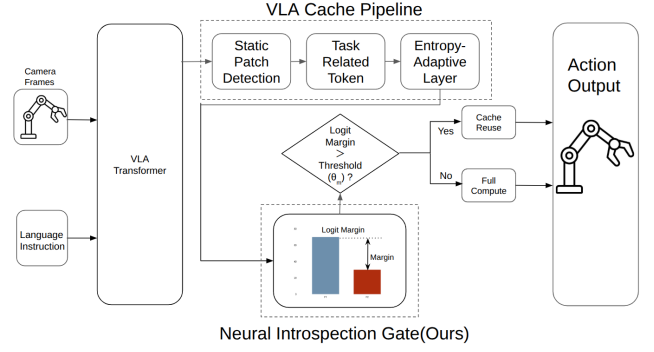


Fig. 2: **Gated VLA-Cache architecture.** At each step t , the logit-margin gate evaluates the action confidence from step $t-1$. If the margin signals low confidence, the cache is invalidated and full inference runs for step t . Otherwise, the standard VLA-Cache pipeline proceeds.

indicates whether a layer spreads attention broadly or focuses it narrowly. VLA-Cache computes per-layer normalized attention entropy and assigns each layer a different proportion of tokens to reuse: low-entropy layers reuse more cached tokens, while high-entropy layers recompute more, with weighted growth smoothing to prevent abrupt transitions.

IV. GATED VLA-CACHE

VLA-Cache’s three stages operate entirely in observation space; they examine what the input looks like but not how the model responds to it. Gated VLA-Cache adds a complementary *gating module* that inspects the model’s own action-prediction confidence to decide whether the cache should be trusted (Fig. 2).

A. Logit-Margin Gating

During autoregressive decoding of the D action tokens at step $t-1$, the model produces a softmax distribution over the vocabulary for each action dimension. We compute the **mean top-1/top-2 probability margin**:

$$m_{t-1} = \frac{1}{D} \sum_{d=1}^D [p_1(a_{t-1}^d) - p_2(a_{t-1}^d)], \quad (1)$$

where p_1 and p_2 denote the highest and second-highest probabilities in the softmax distribution for each action token.

A large margin indicates a confident prediction; a small margin means the model is unsure about competing actions, and choosing incorrectly can degrade performance. Algorithm 1 summarizes the full inference procedure. When the cache is invalidated, the model runs a complete forward pass as if no cache existed, producing fresh KV representations for all tokens. These fresh representations then save the cache for subsequent steps. When the margin is above θ_m , the standard VLA-Cache pipeline executes with its three-stage reuse mechanism.

Algorithm 1 Gated VLA-Cache Inference at Step t

Require: Image I_t , instruction ℓ , KV cache from step $t-1$, margin threshold θ_m

Ensure: Action a_t , updated KV cache

- 1: Compute margin m_{t-1} from step $t-1$ output scores (Eq. 1)
- 2: **if** $m_{t-1} < \theta_m$ **then**
- 3: Invalidate entire KV cache
- 4: Run full forward pass on all $N + |\ell|$ tokens
- 5: Store fresh KV representations in cache
- 6: **else**
- 7: Detect static patches via cosine similarity (*Stage 1*)
- 8: Filter task-relevant patches (*Stage 2*)
- 9: Apply entropy-adaptive layer-wise reuse (*Stage 3*)
- 10: Run partial forward pass on non-cached tokens
- 11: **end if**
- 12: Decode action tokens $a_t = (a_t^1, \dots, a_t^D)$ autoregressively
- 13: **return** a_t , updated cache

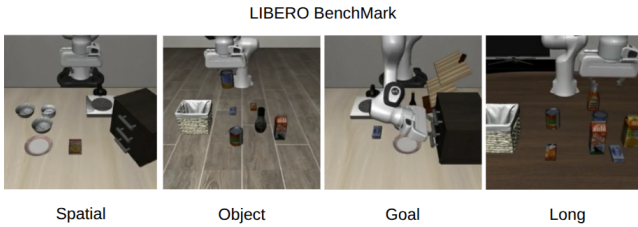


Fig. 3: Tasks on LIBERO Benchmark

B. Properties of the Gating Signal

One-step reactive lag. The margin from step $t-1$ determines cache policy at step t .

Self-regulating behavior. If cache reuse is safe, the margin stays high and the gate never fires, so the method reduces to standard VLA-Cache with negligible overhead. However, if caching produces uncertain actions, the margin drops below the threshold and triggers a recompute. The gate activates only when needed, with no prior knowledge of the task difficulty needed.

V. EXPERIMENTS

A. Experimental Setup

Models. We evaluate on two VLA architectures: (a) OpenVLA [1], a 7B-parameter VLA based on the Prismatic VLM architecture with a Llama-2 backbone, and (b) OpenVLA-OFT [5], which uses optimized fine-tuning with parallel action-dimension decoding for improved downstream performance.

Benchmark. We evaluate on four LIBERO [22] task families: *Spatial* (10 spatial relationship reasoning tasks), *Object* (10 object type identification tasks), *Goal* (10 goal-conditioned tasks), and *Long* (10 long-horizon multi-step tasks). Following the standard evaluation protocol, we run 50 episodes per task (500 episodes per suite). All experiments are conducted on an NVIDIA A100 GPU. In this research work, we used the UTokyo Azure [23].

TABLE I: **Task success rates (%) and compute cost (TFLOPs/step) on LIBERO with OpenVLA.** Gated VLA-Cache recovers accuracy lost by VLA-Cache’s blind caching on tasks where caching causes degradation (Goal, Long), while retaining most of the computational savings. Best success rates are **bolded**.

Task	Full		VLA-Cache		Ours	
	SR%	TFLOP	SR%	TFLOP	SR%	TFLOP
Spatial	78.8	1.89	78.8	1.43	79.4	1.55
Object	70.4	1.86	69.4	1.44	67.8	1.54
Goal	77.2	1.83	74.0	1.40	77.4	1.50
Long	54.0	1.88	50.2	1.43	54.8	1.54
Avg.	70.1	1.87	68.1	1.43	69.9	1.53

Baselines. We compare: (a) **Full Inference**, the OpenVLA model and OpenVLA-OFT model (8 action chunks for default setting) with official weights for comparability; (b) **VLA-Cache** [4], the observation-heuristic caching baseline with default hyperparameters ($\tau=0.996$, $\text{top-}k=120$).

Metrics. We report task success rate (%) and per-step computational cost in TFLOPs.

Hyperparameters. The optimal margin scale depends on the specific VLA architecture’s output calibration. We perform a coarse grid search to find a single threshold θ_m per architecture that provides the best accuracy in LIBERO-Spatial ($\theta_m = 0.65$ for OpenVLA; $\theta_m = 0.50$ for OpenVLA-OFT). This single value is then fixed for all tasks within that specific model architecture.

B. Main Results: OpenVLA

Table I summarizes the results with OpenVLA across four LIBERO suites (500 episodes each). The effect of Gated VLA-Cache depends on how much accuracy VLA-Cache loses on each suite:

Strong recovery on challenging tasks. On LIBERO-Goal, VLA-Cache degrades accuracy by 3.2% (77.2%→74.0%); Gated VLA-Cache recovers 106% of this gap (77.4%), slightly exceeding full-inference performance. On LIBERO-Long, a long-horizon multi-step suite, VLA-Cache drops 3.8% (54.0%→50.2%); Gated VLA-Cache recovers 121% of this gap (54.8%), *exceeding* full-inference accuracy by 0.8%. We suspect this comes from temporal smoothing under selective cache reuse: on confident steps, cached representations reduce noise from frame-to-frame variations.

Neutral when caching is already safe. On LIBERO-Spatial, where VLA-Cache causes no degradation (both at 78.8%), Gated VLA-Cache achieves 79.4% (+0.6%), since we tuned the hyperparameter θ_m on this suite. On LIBERO-Object, where caching causes only a 1.0% drop, Gated VLA-Cache scores 67.8%, slightly below both baselines. This likely reflects occasional unnecessary cache invalidation on tasks with short manipulation horizons where caching is consistently safe.

TABLE II: **Task success rates (%) and compute cost (TFLOPs/step) on LIBERO with OpenVLA-OFT.** OFT achieves high accuracy (93–98%) across all suites. All three methods perform comparably, confirming that Gated VLA-Cache introduces no harm when caching is already safe. Best success rates are **bolded**.

Task	Full		VLA-Cache		Ours	
	SR%	TFLOP	SR%	TFLOP	SR%	TFLOP
Spatial	93.8	4.00	93.8	3.11	94.0	3.23
Object	98.4	3.96	98.6	3.05	98.6	3.08
Goal	97.2	3.94	96.8	3.06	97.0	3.07
Long	93.8	3.98	94.0	3.05	93.0	3.13
Avg.	95.8	3.97	95.8	3.07	95.7	3.13

Average across all suites, Gated VLA-Cache achieves 69.9% at 1.53 TFLOPs, improving over VLA-Cache (68.1%, 1.43 TFLOPs) by 1.8% while remaining 18% below full-inference cost (70.1%, 1.87 TFLOPs).

C. Main Results: OpenVLA-OFT

Table II shows results with OpenVLA-OFT. OFT’s improved fine-tuning produces models that are inherently robust to cache reuse: VLA-Cache causes at most 0.4% degradation on any suite, and in some cases (Object, Long) caching slightly improves accuracy due to its temporal smoothing effect. Consequently, all three methods achieve nearly identical performance ($\pm 0.8\%$), with Gated VLA-Cache improving slightly over VLA-Cache on Spatial and Goal, while tying on Object and dropping slightly on Long.

This result supports the *self-regulating* behavior of Gated VLA-Cache: when cache reuse is already safe (as with OFT), the margin remains high, the gate rarely triggers, and the method adds negligible overhead. In other words, the gate is most useful where OpenVLA struggles under blind caching and mostly dormant otherwise.

D. Accuracy Compute Trade-off

Fig. 4 plots the accuracy compute trade off on the average score of LIBERO Benchmark in OpenVLA. The VLA-Cache causes the most degradation but the least computation. Gated VLA-Cache achieves 69.9% at 1.53 TFLOPs, at 18% below full-inference cost (1.87 TFLOPs) while dropping only 0.2% accuracy. Gated VLA-Cache lies above the linear interpolation between VLA-Cache and Full Inference, making it Pareto-optimal.

E. When Does Gating Fire?

Fig. 5 visualizes the logit-margin signal during a representative LIBERO-Long episode. The margin shows a clear temporal pattern: it stays high during segments where the robot is executing a straightforward motion (e.g., approaching an object), and dips during critical manipulation phases where the action space is genuinely ambiguous (e.g., grasp alignment, transition between sub-tasks). Cache invalidation

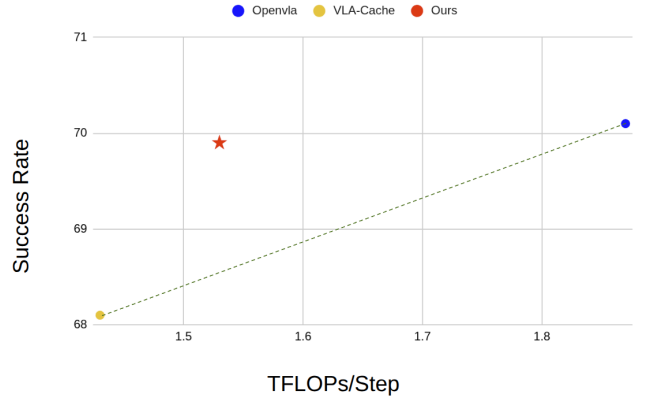


Fig. 4: **Accuracy compute trade off** on average score of four LIBERO Benchmark (OpenVLA). Gated VLA-Cache (star) lies above the line connecting Full Inference and VLA-Cache, indicating Pareto-optimality: it achieves full-inference accuracy at substantially lower compute.

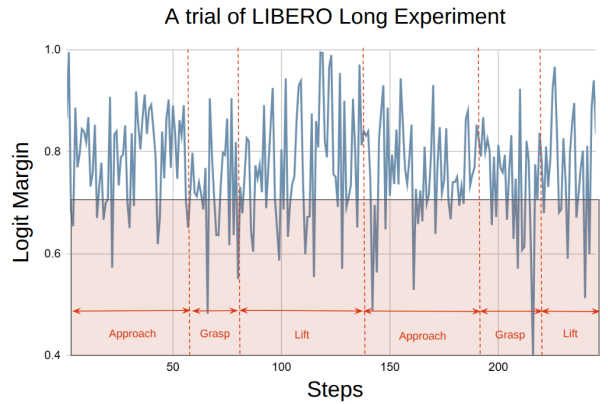


Fig. 5: **Gating trace** for a representative LIBERO-Long episode. The logit margin (blue) dips during critical manipulation phases (grasping objects). Pink regions mark gating events. Each manipulation has three phases: approach, grasp, and lift.

events cluster around these critical moments. The gate fires where fresh representations matter most.

On average across LIBERO-Long episodes, the gate fires on approximately 24% of steps: Gated VLA-Cache reuses cached representations 76% of the time, triggering full recompute only when the model’s confidence drops.

F. Threshold Sensitivity

Fig. 6 shows the effect of varying the margin threshold θ_m . Success rate is stable across a broad range: sweeping θ_m from 0.55 to 0.75 on LIBERO-Spatial yields success rates tightly bounded between 78.2% and 79.4%. At very low thresholds ($\theta_m \rightarrow 0$), the gate never fires and the method reduces to VLA-Cache. Conversely, at very high thresholds, the gate fires too often, causing unnecessary invalidation that can disrupt the temporal smoothing benefits of caching and slightly underperform the baseline (e.g., 78.2% at $\theta_m =$

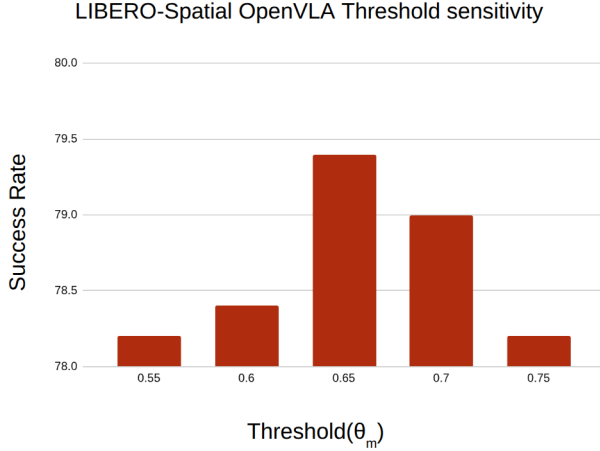


Fig. 6: **Threshold sensitivity** on LIBERO-Spatial (OpenVLA). The method exhibits a broad plateau of near-optimal performance.

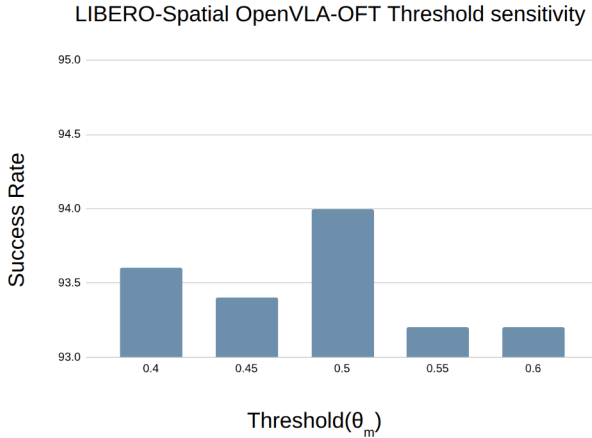


Fig. 7: **Threshold sensitivity** on LIBERO-Spatial (OpenVLA-OFT). The method exhibits a broad plateau of near-optimal performance.

0.75). The chosen threshold ($\theta_m = 0.65$) sits in a stable region that balances accuracy recovery and compute savings.

This robustness transfers to the OpenVLA-OFT architecture as well (Fig. 7). While the calibrated center point shifts due to the different action head formulation ($\theta_m = 0.5$ for OFT), the stability remains: sweeping θ_m from 0.40 to 0.60 on LIBERO-Spatial yields success rates tightly bounded between 93.2% and 94.0%. The gating module does not need precise hyperparameter tuning.

VI. DISCUSSION

When does gating help? The pattern is simple: Gated VLA-Cache helps most on tasks where VLA-Cache causes clear accuracy degradation. On LIBERO-Goal (−3.2%) and LIBERO-Long (−3.8%), where cache staleness compounds over long manipulation sequences, our gating recovers over 100% of the lost accuracy (106% and 121%, respectively).

On suites where caching is already safe (Spatial, Object), the gating rarely triggers and adds minimal overhead. In practice, this means users do not need to predict in advance which tasks will be sensitive to cache reuse. Gated VLA-Cache activates when needed.

Why logit margin? The logit margin captures the model’s output uncertainty: its difficulty in selecting a single best action from the vocabulary. This is more direct than observation-space signals, which only measure what has changed visually without knowing how those changes affect predictions. A visually minor change (e.g., a subtle rotation of a thin object) can dramatically alter the model’s action prediction but go undetected by pixel similarity. The margin reflects this internal effect directly.

In our initial experiments, we evaluated alternative uncertainty signals, including the entropy of the action distribution and attention entropy, but found both ineffective for temporal gating. We suspect that action entropy fails because the model assigns near-zero probability to the vast majority of its large vocabulary (e.g., 32,000 tokens for OpenVLA). In this regime, the mathematical difference in total entropy between a confident prediction (e.g., a 90% vs. 5% split for the top choices) and an unconfident prediction (e.g., a 45% vs. 40% split) is overwhelmed by the static noise of the long-tail vocabulary.

Similarly, attention entropy fails as a temporal gating signal due to the attention sink phenomenon [24] pervasive in deep autoregressive language models. In LLM-based architectures like OpenVLA, the model structurally allocates a massive proportion of attention mass to initial syntactic tokens. Because these sink tokens absorb the majority of the probability mass, the overall entropy of the attention distribution remains artificially anchored throughout an episode. Consequently, meaningful dynamic shifts in visual attention (e.g., shifting focus from the gripper to the target object) barely perturb the total entropy.

We also experimented with the action delta between consecutive steps as a gating signal. However, we found that it primarily correlates with manipulation phase transitions rather than model uncertainty: action deltas are large during approach and grasp phases, and small during post-grasp lifting. This makes it a phase-change indicator, not an uncertainty signal, sharing the same fundamental blind spot as observation-space heuristics.

The top-1/top-2 logit margin avoids both problems. By strictly isolating the probability gap between the two most likely action tokens, it ignores vocabulary noise entirely. It explicitly measures the model’s internal conflict at the bottleneck of decision-making, triggering a recomputation only when the cached representation lacks the discriminatory power to resolve the action safely.

Self-regulating compute. The main advantage of margin-based gating is that the invalidation frequency naturally adapts to task difficulty. On simple tasks (Spatial, Object), the model is consistently confident and the gate rarely fires (<10% of steps), preserving nearly all of VLA-Cache’s speedup. On hard tasks (Goal, Long), the model is frequently

uncertain ($\sim 24\%$ of steps), triggering more recomputation where it is most needed. No task-specific tuning is required.

Cross-architecture generality. We evaluate on two VLA architectures with different action prediction strategies: OpenVLA uses discrete token decoding while OpenVLA-OFT uses parallel action-dimension decoding. The gating signal transfers without modification because it is extracted from the shared transformer backbone’s output logits, which are present in both architectures.

Limitations. (1) The logit margin cannot distinguish between cache-induced uncertainty and inherent model ambiguity. In practice, unnecessary recomputation at difficult moments incurs a minor compute overhead but no accuracy penalty. Invalidation is always safe, just occasionally wasteful. (2) The margin is computed from step $t-1$ and used to decide cache reuse at step t , introducing a one-step reactive lag. (3) The method introduces one hyperparameter (θ_m). While we show it generalizes zero-shot across distinct task suites for a given model, although the threshold is robust (Figs. 6 and 7), its optimal value depends on the specific VLA architecture’s output calibration. Thus, θ_m must be calibrated once per model family (e.g., via grid search on a small validation set). Automatic threshold adaptation (e.g., via online margin statistics) could remove this requirement in future work. (4) We evaluate in simulation. Real robot validation remains future work.

VII. CONCLUSION

We presented Gated VLA-Cache, a lightweight, training-free method for improving KV-cache reuse in Vision-Language-Action models. By monitoring the model’s logit margin, the probability gap between the top-two predicted action tokens, Gated VLA-Cache decides when cached representations are safe to reuse and when a full recompute is needed. On LIBERO with OpenVLA, our method recovers over 100% of the accuracy lost by blind caching on challenging long-horizon and goal-conditioned tasks, while retaining 80% of the computational savings. On OpenVLA-OFT, where caching is already safe, the gate remains dormant and introduces no degradation. The method requires no additional parameters, no retraining, and can be integrated transparently into any autoregressive VLA inference stack.

REFERENCES

- [1] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, *et al.*, “Openvla: An open-source vision-language-action model,” *arXiv preprint arXiv:2406.09246*, 2024.
- [2] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid, *et al.*, “Rt-2: Vision-language-action models transfer web knowledge to robotic control,” in *Conference on Robot Learning*, pp. 2165–2183, PMLR, 2023.
- [3] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, *et al.*, “ π_0 : A vision-language-action flow model for general robot control,” *arXiv preprint arXiv:2410.24164*, 2024.
- [4] S. Xu, Y. Wang, C. Xia, D. Zhu, T. Huang, and C. Xu, “Vla-cache: Efficient vision-language-action manipulation via adaptive token caching,” *arXiv preprint arXiv:2502.02175*, 2025.
- [5] M. J. Kim, C. Finn, and P. Liang, “Fine-tuning vision-language-action models: Optimizing speed and success,” *arXiv preprint arXiv:2502.19645*, 2025.
- [6] S. Karamcheti, S. Nair, A. Balakrishna, P. Liang, T. Kollar, and D. Sadigh, “Prismatic vlms: Investigating the design space of visually-conditioned language models,” in *Forty-first International Conference on Machine Learning*, 2024.
- [7] Y. Lipman, R. T. Chen, H. Ben-Hamu, M. Nickel, and M. Le, “Flow matching for generative modeling,” *arXiv preprint arXiv:2210.02747*, 2022.
- [8] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, *et al.*, “Llama 2: Open foundation and fine-tuned chat models,” *arXiv preprint arXiv:2307.09288*, 2023.
- [9] A. O’Neill, A. Rehman, A. Maddukuri, A. Gupta, A. Padalkar, A. Lee, A. Pooley, A. Gupta, A. Mandlekar, A. Jain, *et al.*, “Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 6892–6903, IEEE, 2024.
- [10] J. Wen, Y. Zhu, J. Li, M. Zhu, K. Wu, Z. Xu, N. Liu, R. Cheng, C. Shen, Y. Peng, *et al.*, “Tinyvla: Towards fast, data-efficient vision-language-action models for robotic manipulation,” in *IEEE Robotics and Automation Letters (RA-L)*, 2025.
- [11] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, “Diffusion policy: Visuomotor policy learning via action diffusion,” *The International Journal of Robotics Research*, vol. 44, no. 10–11, pp. 1684–1704, 2025.
- [12] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti, *et al.*, “Smolvla: A vision-language-action model for affordable and efficient robotics,” *arXiv preprint arXiv:2506.01844*, 2025.
- [13] J. Liu, M. Liu, Z. Wang, P. An, X. Li, K. Zhou, S. Yang, R. Zhang, Y. Guo, and S. Zhang, “Robomamba: Efficient vision-language-action model for robotic reasoning and manipulation,” in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- [14] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, E. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in neural information processing systems*, vol. 30, 2017.
- [15] A. Gu and T. Dao, “Mamba: Linear-time sequence modeling with selective state spaces,” in *First conference on language modeling*, 2024.
- [16] L. Chen, H. Zhao, T. Liu, S. Bai, J. Lin, C. Zhou, and B. Chang, “An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models,” in *European Conference on Computer Vision*, pp. 19–35, Springer, 2024.
- [17] L. Xing, Q. Huang, X. Dong, J. Lu, P. Zhang, Y. Zang, Y. Cao, C. He, J. Wang, F. Wu, *et al.*, “Pyramiddrop: Accelerating your large vision-language models via pyramid visual redundancy reduction,” *arXiv preprint arXiv:2410.17247*, 2024.
- [18] Y. Gal and Z. Ghahramani, “Dropout as a bayesian approximation: Representing model uncertainty in deep learning,” in *international conference on machine learning*, pp. 1050–1059, PMLR, 2016.
- [19] B. Lakshminarayanan, A. Pritzel, and C. Blundell, “Simple and scalable predictive uncertainty estimation using deep ensembles,” *Advances in neural information processing systems*, vol. 30, 2017.
- [20] D. Hendrycks and K. Gimpel, “A baseline for detecting misclassified and out-of-distribution examples in neural networks,” *arXiv preprint arXiv:1610.02136*, 2016.
- [21] B. Settles, “Active learning literature survey,” Computer Sciences Technical Report 1648, University of Wisconsin-Madison, 2009.
- [22] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “Libero: Benchmarking knowledge transfer for lifelong robot learning,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 44776–44791, 2023.
- [23] M. Nakamura, R. Nakamura, K. Arima, K. Sasaki, Y. Yamakata, K. Ito, M. Nishijima, H. Miyazaki, T. Ishizaki, J. Tamatsukuri, T. Amemiya, S. Chiba, and K. Taura, “Utokyo azure: Deployment of a cloud computing service at the university of tokyo,” *Journal for Academic Computing and Networking*, vol. 29, no. 1, pp. 180–186, 2025.
- [24] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, “Efficient streaming language models with attention sinks,” *arXiv preprint arXiv:2309.17453*, 2023.